
Software Requirements Specification

for

Information System for Airline Flight Management

Version 0.3 approved

Prepared by : Ciutureanu Vlad Andrei

Cumpănașu Ștefan-Adrian

Eduard-Lucian Corbeanu

CR 3.1B

27.02.2026

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Document Conventions.....	1
1.3 Intended Audience and Reading Suggestions.....	1
1.4 Product Scope	1
1.5 References.....	1
2. Overall Description	2
2.1 Product Perspective.....	2
2.2 Product Functions	2
2.3 User Classes and Characteristics	2
2.4 Operating Environment.....	3
2.5 Design and Implementation Constraints	3
2.6 User Documentation	3
2.7 Assumptions and Dependencies	3
3. External Interface Requirements	4
3.1 User Interfaces	4
3.2 Hardware Interfaces	4
3.3 Software Interfaces	4
3.4 Communications Interfaces	4
4. System Features	4
4.1 System Architecture and Security.....	5
4.2 Passenger Management	6
4.3 Flight Management.....	6
4.4 Reservation and Ticketing.....	7
4.5 Payments, Reporting, and UI Behavior.....	7
5. Other Nonfunctional Requirements.....	8
5.1 Performance Requirements.....	8
5.2 Safety Requirements.....	9
5.3 Security Requirements.....	9
5.4 Software Quality Attributes	9
5.5 Business Rules	10
6. Other Requirements	10
6.1 Code Quality Rules.....	10

Revision History

Name	Date	Reason For Changes	Version
The start	27.02.2026	Started working on the project	0.0
Section 1, 2, 4	02.03.2026	Finished sections 1, 2 and 4	0.1
Section 3	04.03.2026	Finished section 3	0.2
Section 5, 6	10.03.2026	Finished sections 5 and 6	0.3

1. Introduction

1.1 Purpose

This document describes the requirements specification for the project: Information System for Airline Flight Management.

1.2 Document Conventions

1.3 Intended Audience and Reading Suggestions

We suggest a specific sequence for reading the document, beginning with the overview sections and proceeding through the sections that are most pertinent to each reader type:

- **All Readers:** Should begin by reading Section 1 (Introduction) and Section 2 (Overall Description) to gain a foundational understanding of the system's purpose and context.
- **Project Managers:** Should focus on Section 1, Section 2, and Section 5 (Other Nonfunctional Requirements) to understand the project scope and operational constraints.
- **Developers:** Should review Section 3 (External Interface Requirements) and Section 4 (System Features) to guide the technical implementation.
- **QA/Testers:** Should focus heavily on Section 4 (System Features) and the Business Rules in Section 5 to design accurate test cases and validation scripts.

1.4 Product Scope

The objective of this project is to develop a software system designed to automate the flight and customer management processes for an airline company, facilitating the following activities:

- Collection of data related to operated flights (including departure airports, arrival airports, and intermediate layover points).
- Management of passenger data.
- Generation of comprehensive reports.
- Provision of information via a centralized web portal.
- Execution of complex system queries.

1.5 References

List of web addresses used as references for the user interface style:

- <https://wizzair.esky.ro>
- <https://www.kiwi.com>
- <https://www.vola.ro>
- <https://www.aa.com>

2. Overall Description

2.1 Product Perspective

The Information System for Airline Flight Management is a new, self-contained software product. It is designed to replace manual document workflows and fragmented legacy systems with a centralized, automated platform for managing flights, passengers, and reservations.

While it is a self-contained product, it operates as a distributed system comprised of three main interacting components:

1. **The Client Subsystem (Frontend):** A web-based user interface accessed via standard web browsers, built with HTML, CSS, and JavaScript. It serves as the portal for both airline staff and customers.
2. **The Application Server Subsystem (Backend):** Built on the ASP.NET Core MVC framework, this layer handles all the business logic, security (authentication/authorization), reporting, and processing of reservations.
3. **The Database Subsystem:** A relational database management system (RDBMS) responsible for securely storing, backing up, and retrieving large volumes of data regarding flight schedules, passenger profiles, and financial transactions.

2.2 Product Functions

The Distributed Information System for Airline Flight Management provides a comprehensive set of tools for both airline personnel and customers. The major functions of the system are organized into the following high-level categories:

- **User Authentication and Security:** Allows users to register, log in securely, and interact with the web portal based on their assigned roles (e.g., Administrator, Staff, Customer).
- **Flight Management:** Enables airline administrators to schedule, update, and cancel flights, as well as assign aircraft types and manage seating capacities.
- **Passenger Management:** Allows for the collection, storage, and modification of passenger profile data (name, contact information, address) through secure web forms.
- **Reservation and Ticketing (Document Workflow):** Allows customers to search for available flights, select booking classes, and reserve seats. The system automatically manages seat availability and tracks the booking lifecycle.

2.3 User Classes and Characteristics

The system distinguishes between three primary user classes based on their security privilege levels and functional needs:

- **Customer (Passenger):** The most frequent and critical user class for the system's success. Customers require no specialized technical expertise. Their access is strictly limited to

managing their personal profiles, searching for flights, executing payments, and handling their own reservations.

- **Staff (Airline Personnel):** Users responsible for day-to-day operations. They require intermediate privileges to view passenger lists, generate occupancy reports. They cannot alter system-wide settings or core flight schedules.
- **Administrator:** The most restricted group in terms of numbers, but possessing the highest security privileges. Administrators are responsible for core system maintenance, such as creating or canceling flights, managing aircraft data, and assigning access rights to other users (including Staff).

2.4 Operating Environment

The system operates in a distributed client-server architecture, divided into three environments:

- **Client Environment:** Any internet-enabled device running a modern, HTML5/CSS3/JavaScript-compliant web browser (e.g., Chrome, Safari, Edge). No additional desktop software or plugins are required.
- **Application Server (Backend):** A Windows or Linux server running the .NET Runtime to host the ASP.NET Core MVC application, utilizing a standard web server like IIS or Kestrel.
- **Database Server:** A dedicated relational database management system (RDBMS) such as Microsoft SQL Server supported by Entity Framework Core.
- **Network:** Components must communicate over a standard TCP/IP network, using HTTP/HTTPS for web traffic and standard secure ports for database connections.

2.5 Design and Implementation Constraints

2.6 User Documentation

2.7 Assumptions and Dependencies

3. External Interface Requirements

3.1 User Interfaces

The user interface (UI) for the Information System for Airline Flight Management shall be a responsive, web-based application built using HTML5, CSS3, and JavaScript.

- **Design Standards:** The UI shall follow modern web design principles. Reference styles include existing airline platforms such as WizzAir and Kiwi.com.
- **Screen Layout:** Every screen shall feature a persistent top navigation bar (for accessing Flights, Reservations, Profile, and Login/Logout) and a standard footer containing contact information and legal disclaimers.
- **Error Handling:** Client-side JavaScript validation shall intercept incomplete or invalid form submissions before sending data to the server, displaying clear, localized error messages adjacent to the relevant input fields.
- **Notifications:** The system shall display clear success notifications (e.g., "Booking Successful") following critical user actions.

3.2 Hardware Interfaces

As a web-based application, the system does not require direct interaction with specialized custom hardware.

3.3 Software Interfaces

The system relies on the following major software interfaces to function:

- **Operating Systems:** The Application Server shall be hosted on a Windows or Linux server utilizing the .NET Runtime.
- **Database Management System (DBMS):** The ASP.NET Core backend shall communicate with a relational database (Microsoft SQL Server) via Entity Framework Core. This interface will handle data reading, writing, and backup executions.
- **External Flight API:** The backend shall interface with a third-party Flight API (Amadeus). The system will send search parameters (Source, Destination, Date) and consume the incoming flight schedule data formatted in JSON.

3.4 Communications Interfaces

The system relies on the following communication protocols to ensure secure and efficient data exchange between its core components:

- **Client-to-Server:** All interactions between the web browser and the ASP.NET Core application server shall occur over a TCP/IP network using HTTP and HTTPS protocols. HTTPS will ensure the secure, encrypted transmission of sensitive passenger profile and payment data.

- **Server-to-Database:** The application server shall communicate with the Microsoft SQL Server database using standard secure database connection strings and dedicated network ports (TCP Port 1433).
- **Server-to-API:** The backend shall communicate with the external Flight API (Amadeus) via RESTful HTTP/HTTPS requests, sending search parameters and receiving parsed JSON payloads containing the real-time flight data.

4. System Features

This section details the specific functional requirements of the Information System for Airline Flight Management. The requirements are organized by major system features, which represent the primary services provided by the software. Each feature subsection includes a brief description, a priority level, typical stimulus/response sequences to illustrate user interactions, and the granular, verifiable functional requirements necessary for the development team to implement the ASP.NET Core backend and web interface.

4.1 System Architecture and Security

4.1.1 Description and Priority This feature covers the underlying distributed infrastructure, database administration, and role-based access control. **Priority:** High.

4.1.2 Stimulus/Response Sequences

- **Stimulus:** An unregistered user attempts to access the portal. **Response:** The system redirects the user to the secure login/registration page.
- **Stimulus:** A user submits valid registration details. **Response:** The system creates the account, assigns default privileges, and logs the user in.

4.1.3 Functional Requirements

- **REQ-1:** The system shall utilize a distributed architecture, including a Web Server, an Application Server, and a Database Server.
- **REQ-2:** The user interface shall be accessible via standard HTML5 web browsers without additional plugins.
- **REQ-3:** The system shall include a secure login module for user authentication.
- **REQ-4:** The system shall enforce Role-Based Access Control (RBAC) to differentiate access between Administrators, Staff, and Customers.
- **REQ-5:** The system shall allow new users to register an account via the web interface.
- **REQ-6:** The system shall automatically assign a predefined set of default access rights to newly registered users.
- **REQ-7:** The system shall allow an Administrator to modify the access rights of existing users.
- **REQ-8:** The Database Management System (DBMS) shall support offline backup operations.
- **REQ-9:** The DBMS shall support point-in-time data recovery.
- **REQ-10:** The DBMS shall permit administration and monitoring via a web browser.
- **REQ-11:** The system shall validate email address formats upon registration and profile updates.
- **REQ-12:** The system shall enforce secure password policies (minimum length, alphanumeric constraints) during account creation.

4.2 Passenger Management

4.2.1 Description and Priority This feature governs the collection, storage, and modification of client profile data. **Priority:** High.

4.2.2 Stimulus/Response Sequences

- **Stimulus:** A customer updates their contact information in the portal. **Response:** The system validates the input and updates the passenger record in the database.
- **Stimulus:** An administrator requests the client list. **Response:** The system retrieves the passenger data and displays it in a sortable, tabular format.

4.2.3 Functional Requirements

- **REQ-13:** The system shall securely store passenger personal data.
- **REQ-14:** A passenger profile shall mandatorily include: First Name, Last Name, Phone Number, Email, and Address.
- **REQ-15:** The system shall allow an authenticated user to view and modify their own contact details.
- **REQ-16:** The system shall collect passenger data exclusively through secure web forms.
- **REQ-17:** The system shall display client lists for administrators and staff in a tabular format.
- **REQ-18:** The system shall allow an Administrator to logically delete (deactivate) a passenger account.
- **REQ-19:** The system shall prevent the registration of multiple accounts using the same email address.

4.3 Flight Management

4.3.1 Description and Priority This feature enables the creation, modification, and tracking of airline flight schedules and aircraft capacities. **Priority:** High.

4.3.2 Stimulus/Response Sequences

- **Stimulus:** A customer searches for flights from Airport A to Airport B on a specific date. **Response:** The system sends an HTTP request to the external Flight API, parses the JSON response, and displays the available flights.
- **Stimulus:** A customer selects a flight to book. **Response:** The system caches (saves) that specific flight's details into the local database so the booking record remains stable even if the API goes offline.

4.3.3 Functional Requirements

- **REQ-20:** The system shall integrate with a third-party Flight API to retrieve real-time flight schedules, eliminating the need for manual flight data entry.
- **REQ-21:** The system shall utilize the unique Flight Identifier provided by the external API for all internal booking references.
- **REQ-22:** The system shall allow users to search for flights by inputting a Source (departure) and Destination (arrival) location.
- **REQ-23:** The system shall display the scheduled Departure Time, Arrival Time, and Airline name precisely as provided by the API response.
- **REQ-24:** The system shall locally cache (store in the SQL database) the details of any flight for which a reservation is initiated, ensuring data integrity for the document workflow.
- **REQ-25:** The system shall verify flight availability status via the API immediately before finalizing a customer's reservation.
- **REQ-26:** The system shall display the API flight search results in a structured, tabular format on the web portal.

- **REQ-27:** The system shall allow an Administrator to configure and update the Flight API connection credentials via a secure admin panel.
- **REQ-28:** The system shall log all failed API requests (e.g., timeouts, 404 errors) to the database for administrative troubleshooting.
- **REQ-29:** The system shall automatically notify the user and halt the booking process if the external API reports the selected flight has been canceled or is fully booked.

4.4 Reservation and Ticketing

4.4.1 Description and Priority This feature handles the core business logic of associating passengers with flights and managing seat availability. **Priority:** High.

4.4.2 Stimulus/Response Sequences

- **Stimulus:** A passenger confirms a flight booking. **Response:** The system generates a reservation record and decreases the flight's available seat count by one.
- **Stimulus:** A passenger cancels their reservation. **Response:** The system updates the reservation status and increases the flight's available seat count by one.

4.4.3 Functional Requirements

- **REQ-30:** The system shall allow an authenticated passenger to purchase tickets for one or more flights.
- **REQ-31:** The system shall associate each reservation with a single flight and a single passenger.
- **REQ-32:** The system shall dynamically decrease the number of available seats for a flight upon reservation confirmation.
- **REQ-33:** The system shall record the timestamp of when a reservation is created.
- **REQ-34:** The system shall require the selection of a flight class (e.g., Economy, Business) during the booking process.
- **REQ-35:** The system shall allow Staff and Administrators to view the passenger manifest (all reservations) for a specific flight.
- **REQ-36:** The system shall allow a passenger to view their own reservation history.
- **REQ-37:** The system shall reject booking requests if the available seat count for the selected flight is zero.
- **REQ-38:** The system shall automatically calculate the total price of a reservation based on the flight and selected class.
- **REQ-39:** The system shall prevent the deletion of a flight from the database if active reservations exist for it.
- **REQ-40:** The system shall allow a passenger to cancel their reservation prior to the flight's departure date.
- **REQ-41:** The system shall release the reserved seat back into the available pool upon reservation cancellation.

4.5 Payments, Reporting, and UI Behavior

4.5.1 Description and Priority This feature covers the processing of financial transactions, the generation of administrative reports, and the frontend user experience. **Priority:** Medium.

4.5.2 Stimulus/Response Sequences

- **Stimulus:** A customer submits payment details for a pending reservation. **Response:** The system records the transaction date, generates a transaction ID, and marks the reservation as paid.
- **Stimulus:** A user submits an incomplete web form. **Response:** The client-side interface intercepts the submission and displays a clear error message.

4.5.3 Functional Requirements

- **REQ-42:** The system shall allow the customer to select a preferred payment method from a predefined list.
- **REQ-43:** The system shall record the actual date the payment transaction is completed.
- **REQ-44:** The system shall generate customized reports accessible based on the user's role (e.g., financial reports for Administrators).
- **REQ-45:** The system shall update the reservation status to "Confirmed and Paid" upon successful payment registration.
- **REQ-46:** The system shall allow an Administrator to generate a flight occupancy report (percentage of seats sold).
- **REQ-47:** The system shall generate a unique Transaction ID for every completed payment for auditing purposes.
- **REQ-48:** The web interface shall utilize client-side scripting (JavaScript) to validate all form inputs before server submission.
- **REQ-49:** The system shall display clear, user-friendly error messages for invalid actions (e.g., "Missing required field").
- **REQ-50:** The system shall display visible success notifications (e.g., "Booking Successful") upon the completion of key actions.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- **General UI Response Time:** To ensure a smooth user experience, the client-side web application (HTML/CSS/JS) shall load and render static pages and forms in less than 2 seconds over a standard broadband.
- **External API Latency:** Because the system interfaces with a third-party Flight API, flight search results based on user parameters (Source, Destination, Date) shall be displayed within a maximum of 5 seconds. If the external API fails to respond within this timeframe, the system shall abort the request, display a user-friendly error message, and log the timeout event to the database.
- **Transaction Processing (Booking):** The critical action of confirming a reservation, which involves generating the record and dynamically decreasing the available seat count for the flight, shall be completed in under 3 seconds to guarantee data integrity and prevent booking collisions.
- **Database Performance:** The relational database (Microsoft SQL Server) shall execute complex administrative queries, such as the flight occupancy reports generated by Staff or Administrators, in less than 10 seconds without locking the tables required for active passenger bookings.
- **System Concurrency and Throughput:** The Application Server (ASP.NET Core MVC) shall concurrently support a minimum of 500 active user sessions without the average server response time degrading beyond 3 seconds.

5.2 Safety Requirements

- **Data Loss Prevention:** The system shall guarantee that no passenger profile or reservation data is permanently lost in the event of a sudden Application Server failure. All booking transactions must be fully ACID-compliant (Atomicity, Consistency, Isolation, Durability) within the Microsoft SQL Server database.
- **Double-Booking Prevention (Concurrency Control):** To prevent the operational harm of overbooking or assigning the same seat to multiple passengers, the Database Subsystem must implement strict row-level locking during the reservation process.
- **Transaction Rollback (Payment Safety):** If a network interruption or system failure occurs exactly during the payment processing phase, the system shall automatically roll back the entire transaction. This safeguard ensures that a customer is never charged for an unconfirmed ticket.
- **External API Fail-Safe:** If the third-party Flight API transmits corrupted data or goes offline, the system shall gracefully degrade by halting new external flight searches and preventing new bookings, displaying a "Service Temporarily Unavailable" message to avoid the creation of unverified reservations.

5.3 Security Requirements

- **Data Encryption in Transit:** All data exchanged between the client web browser and the ASP.NET Core Application Server shall be encrypted using TLS/SSL protocols (HTTPS) to prevent eavesdropping.
- **Payment Security (PCI-DSS):** To ensure financial security, the system shall not store full credit card numbers or sensitive authentication data in the local Microsoft SQL Server database.
- **Authentication and Password Storage:** User authentication shall be managed via secure, token-based mechanisms for API communication. Passwords must never be stored in plain text; they must be cryptographically hashed and salted using industry-standard algorithms before being saved to the database.
- **Vulnerability Prevention:** The Application Server and Database Subsystem must be protected against common web vulnerabilities. Specifically, the use of Entity Framework Core shall be enforced to parameterize database queries, effectively preventing SQL Injection attacks.

5.4 Software Quality Attributes

- **Usability:** For the client-facing web interface, ease of use is prioritized over ease of learning. The flight search and ticket booking workflow must be highly intuitive, allowing a first-time customer to successfully complete a reservation in under 5 minutes without needing to consult any help documentation.
- **Maintainability:** The C# backend codebase shall strictly adhere to the ASP.NET Core MVC (Model-View-Controller) architectural pattern to guarantee a clear separation of concerns.
- **Interoperability:** The system shall maintain seamless interoperability with external services. It must strictly utilize RESTful HTTP/HTTPS standards to consistently parse incoming JSON payloads from the Amadeus Flight API.
- **Testability:** The backend architecture shall be designed to support automated testing. At least 80% of the core business logic must be verifiable via automated unit tests using a standard .NET testing framework.

5.5 Business Rules

- **BR-1 (Overbooking Prevention):** The system shall not allow the total number of confirmed reservations for a specific flight to exceed the maximum seating capacity of the assigned aircraft.
- **BR-2 (Cancellation Window):** A Customer (Passenger) is permitted to independently cancel their confirmed reservation only if the cancellation request is submitted at least 24 hours prior to the scheduled flight departure time.
- **BR-3 (Administrative Overrides):** Only users with the "Administrator" role are authorized to manually cancel a scheduled flight, update API credentials, or modify an existing flight's base price. Users with the "Staff" role are restricted to read-only access regarding flight schedules and cannot alter them.
- **BR-4 (Duplicate Booking Restriction):** A single passenger profile (uniquely identified by their registered email address) cannot hold more than one active reservation for the exact same Flight ID.

6. Other Requirements

6.1 Code Quality Rules

- **Single Responsibility Principle (SRP):** Every class, module, and method should have one, and only one, reason to change.
- **Don't Repeat Yourself (DRY):** Avoid duplicating code across the application.
- **Follow Meaningful Naming Conventions:** Variables, methods, and classes must have clear, descriptive names that reveal their intent.
- **Strict Separation of Concerns (MVC):** The application must strictly enforce the Model-View-Controller architecture.
- **Avoid "Magic" Numbers and Strings:** Hardcoded numbers or strings with unexplained meaning must be avoided.
- **Implement Comprehensive Error Handling:** Use appropriate try-catch blocks for operations that may fail.
- **Ensure Secure Data Access:** All database interactions must use Entity Framework Core's LINQ or parameterized queries to natively prevent SQL Injection vulnerabilities.
- **Write Self-Documenting Code and Use Comments Sparingly:** Code should be written clearly enough to be understood without comments.
- **Keep Methods and Functions Short:** Methods should be concise and focused on a single task.
- **Maintain Consistent Formatting:** All team members must use consistent code formatting.